



**UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE**

**Departamento de Ciencias de la Computación**

**Carrera de Ingeniería de Software**

**Asignatura: Análisis y Diseño de Software – NRC: 30723**

**Tema: Patrones Comportamentales ITERATOR y MEDIATOR**

**Grupo Número: 7 Integrantes:**

**Diana Guerra  
Paul Moreno  
Leonel Tipan**

**Fecha: 21 de mayo del 2026**

---

Sangolquí – Ecuador

**Patrón Iterator**

# Patron Mediator

**Nombre del Patron:** Mediator

**Definición:** El patrón Mediator que permite reducir las dependencias entre objetos. Este patrón restringe las comunicaciones directas entre los distintos componentes, forzándolos a colaborar e interactuar únicamente a través de un objeto central o mediador.

**Descripción del problema:** Se cuenta con una interfaz gráfica para gestionar la información. Esta pantalla se compone de varios controles visuales: botones para Agregar, Actualizar y Eliminar, una tabla para "Mostrar todo" y campos de texto para ingresar el ID, Nombre y Edad del estudiante. A medida que el formulario incorpora la lógica de los requisitos funcionales, las relaciones entre estos elementos se vuelven complejas y caóticas. Por ejemplo:

- Al seleccionar un estudiante en la tabla, los campos de texto deben rellenarse automáticamente con su ID, Nombre y Edad.
- Al hacer clic en el botón "Agregar estudiante", el sistema debe primero invocar la validación de los datos revisando los campos de texto. Si es exitoso, debe actualizar la tabla para mostrar el nuevo registro y luego limpiar los campos.

## Ventajas

- **Principio de responsabilidad única:** Permite extraer las complejas comunicaciones entre los distintos elementos del formulario a un único lugar (el mediador), haciendo el código mucho más fácil de comprender y mantener.
- **Reutilización de componentes:** Al aplicar este patrón, los componentes individuales ya no conocen ni dependen de los otros controles de la interfaz, lo que permite utilizarlos en otras pantallas de la aplicación simplemente vinculándolos a un nuevo mediador.
- **Reducción de acoplamiento:** Minimiza significativamente las dependencias entre los componentes gráficos.

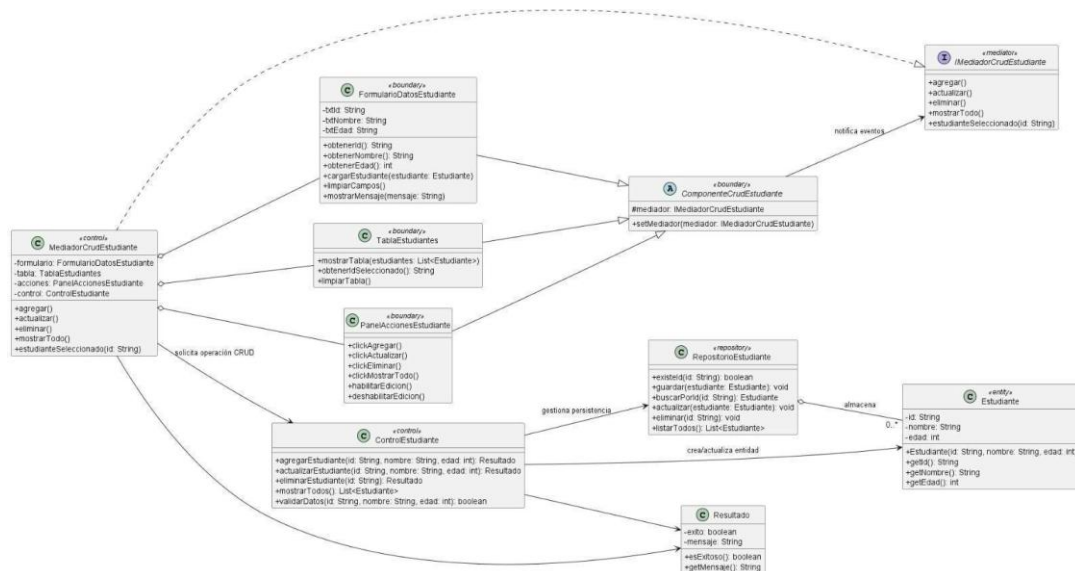
## Desventajas

- **El anti-patrón del Objeto Todopoderoso ("God Object"):** Con el paso del tiempo y a medida que el CRUD crezca en funcionalidades o controles, el objeto MediatorCRUDEstudiantes corre el riesgo de acumular demasiada lógica, evolucionando hasta convertirse en un objeto todopoderoso y monolítico que sea complejo de gestionar

## ¿Por qué usar el patrón Mediator?

| Elemento del patrón Mediator   | Clase en el proyecto      | Responsabilidad                                                                                                                                       |
|--------------------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mediator                       | IMediadorCrudEstudiante   | Define las operaciones que centralizan la comunicación del CRUD: agregar(), actualizar(), eliminar(), mostrarTodo() y estudianteSeleccionado().       |
| Concrete Mediator              | MediadorCrudEstudiante    | Coordina la comunicación entre formulario, tabla, panel de acciones y controlador. Ejecuta las operaciones CRUD según los eventos de la interfaz.     |
| Colleague /<br>Componente base | ComponenteCrudEstudiante  | Clase base para los componentes visuales. Guarda la referencia al mediador y permite que los componentes notifiquen eventos sin depender entre ellos. |
| Concrete Colleague             | FormularioDatosEstudiante | Maneja los campos de ID, nombre y edad. Obtiene datos, carga estudiantes seleccionados, limpia campos y muestra mensajes.                             |
| Concrete Colleague             | TablaEstudiantes          | Muestra la lista de estudiantes, permite obtener el ID seleccionado y limpia la tabla cuando es necesario.                                            |
| Concrete Colleague             | PanelAccionesEstudiante   | Representa los botones de acción: agregar, actualizar, eliminar y mostrar todo. Notifica al mediador cuando ocurre un clic.                           |
| Control de apoyo               | ControlEstudiante         | Ejecuta la lógica de negocio del CRUD y valida los datos antes de enviar respuestas al mediador.                                                      |
| Repositorio de apoyo           | RepositorioEstudiante     | Gestiona las operaciones de almacenamiento: guardar, buscar, actualizar, eliminar y listar estudiantes.                                               |

#### Modelado Mediator:



@startuml

Diagrama Mediator\_CRUD\_Estudiante skinparam

classAttributeIconSize 0 skinparam

shadowing false left to right direction

```

    interface "IMediadorCrudEstudiante" <<mediator>>
    {
        + agregar()
        + actualizar()
        + eliminar()
        + mostrarTodo()
        + estudianteSeleccionado(id: String) }
    abstract class "ComponenteCrudEstudiante"
    <<boundary>> {
        # mediador: IMediadorCrudEstudiante
        + setMediator(mediador: IMediadorCrudEstudiante)
    } class "FormularioDatosEstudiante" <<boundary>>
    {
        - txtId: String
        - txtNombre: String
        - txtEdad: String
        + obtenerId(): String
        + obtenerNombre(): String
        + obtenerEdad(): int
        + cargarEstudiante(estudiante: Estudiante)
        + limpiarCampos()
        + mostrarMensaje(mensaje: String) } class
    "TablaEstudiantes" <<boundary>> { +
        mostrarTabla(estudiantes: List<Estudiante>)
        + obtenerIdSeleccionado(): String
        + limpiarTabla()
    } class "PanelAccionesEstudiante"
    <<boundary>> {
        + clickAgregar()
        + clickActualizar()
        + clickEliminar()
        + clickMostrarTodo()
        + habilitarEdicion()
        + deshabilitarEdicion()
    } class "MediatorCrudEstudiante"
    <<control>> {

```

```

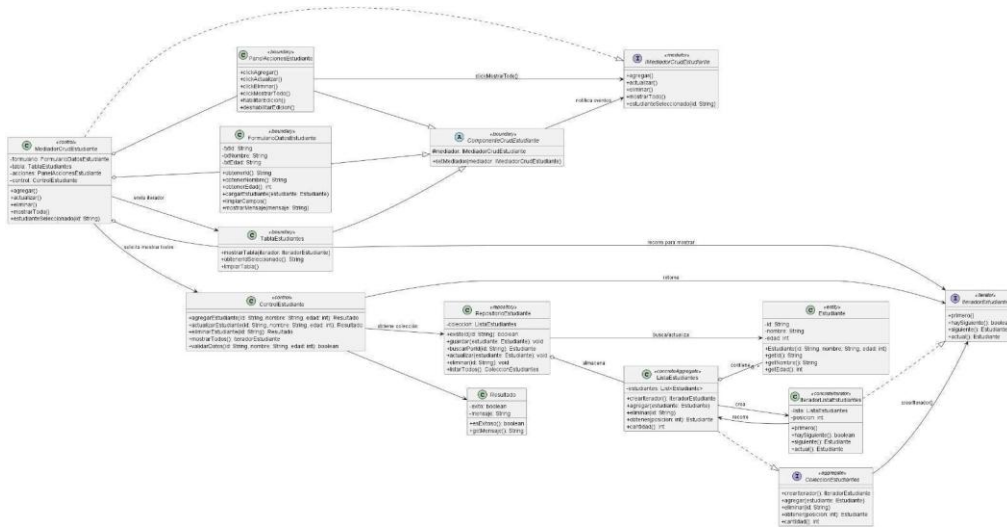
- formulario: FormularioDatosEstudiante
- tabla: TablaEstudiantes
- acciones: PanelAccionesEstudiante - control: ControlEstudiante
  + agregar()
  + actualizar()
  + eliminar()
  + mostrarTodo()
  + estudianteSeleccionado(id: String) }
class "ControlEstudiante"
<<control>> {
  + agregarEstudiante(id: String, nombre: String, edad: int): Resultado
  + actualizarEstudiante(id: String, nombre: String, edad: int): Resultado
  + eliminarEstudiante(id: String): Resultado
  + mostrarTodos(): List<Estudiante>
  + validarDatos(id: String, nombre: String, edad: int): boolean
} class "RepositorioEstudiante"
<<repository>> {
  + existeId(id: String): boolean
  + guardar(estudiante: Estudiante): void
  + buscarPorId(id: String): Estudiante +
actualizar(estudiante: Estudiante): void +
eliminar(id: String): void + listarTodos():
List<Estudiante>
} class "Estudiante" <<entity>>
{
- id: String
- nombre: String
- edad: int
  + Estudiante(id: String, nombre: String, edad: int)
  + getId(): String
  + getNombre(): String
  + getEdad(): int
} class

"Resultado" {
- exito: boolean
- mensaje: String
  + esExitoso(): boolean
  + getMensaje(): String
}

FormularioDatosEstudiante --|> ComponenteCrudEstudiante TablaEstudiantes -
-|> ComponenteCrudEstudiante PanelAccionesEstudiante -|>
ComponenteCrudEstudiante
MediadorCrudEstudiante ..|> IMediadorCrudEstudiante ComponenteCrudEstudiante
--> IMediadorCrudEstudiante : notifica eventos
MediadorCrudEstudiante o-- FormularioDatosEstudiante
MediadorCrudEstudiante o-- TablaEstudiantes
MediadorCrudEstudiante o-- PanelAccionesEstudiante MediadorCrudEstudiante
--> ControlEstudiante : solicita operación CRUD
ControlEstudiante --> Estudiante : crea/actualiza entidad
ControlEstudiante --> RepositorioEstudiante : gestiona persistencia
RepositorioEstudiante o-- "0..*" Estudiante : almacena ControlEstudiante
--> Resultado MediadorCrudEstudiante
--> Resultado
@enduml

```

# Combinación Modelo iterator y mediator



@startuml

Diagrama\_Iterator\_Alineado\_Con\_Mediator

skinparam classAttributeIconSize 0 skinparam

shadowing false left to right direction

```

interface "IMediadorCrudEstudiante" <<mediator>> {
    + agregar()
    + actualizar()
    + eliminar()
    + mostrarTodo()
    + estudianteSeleccionado(id: String)
}

```

```

abstract class "ComponenteCrudEstudiante" <<boundary>> { #
    mediador: IMediadorCrudEstudiante
    + setMediator(mediador:
IMediadorCrudEstudiante)
}

```

```

class "FormularioDatosEstudiante" <<boundary>> {
- txtId: String
- txtNombre: String
- txtEdad: String
    + obtenerId(): String
    + obtenerNombre(): String
    + obtenerEdad(): int + cargarEstudiante(estudiante:
Estudiante)
    + limpiarCampos()
    + mostrarMensaje(mensaje: String)
}

```

```

class "PanelAccionesEstudiante" <<boundary>> {
    + clickAgregar()
    + clickActualizar()
    + clickEliminar()
    + clickMostrarTodo()
+ habilitarEdicion() +
deshabilitarEdicion()
}

```

```

class "TablaEstudiantes" <<boundary>> {
    + mostrarTabla(iterator:
IteratorEstudiante)
}

```

```

    + obtenerIdSeleccionado(): String
    + limpiarTabla()
} class "MediadorCrudEstudiante" <<control>>
{
- formulario: FormularioDatosEstudiante
- tabla: TablaEstudiantes
- acciones: PanelAccionesEstudiante - control: ControlEstudiante
    + agregar()
    + actualizar()
    + eliminar()
    + mostrarTodo()
    + estudianteSeleccionado(id: String)
}
class "ControlEstudiante" <<control>> {
    + agregarEstudiante(id: String, nombre:
String, edad: int): Resultado
                                +
    actualizarEstudiante(id: String, nombre: String,
edad: int): Resultado + eliminarEstudiante(id:
String):
Resultado
    + mostrarTodos(): IteradorEstudiante +
validarDatos(id: String, nombre: String,
edad: int): boolean
}
class "RepositorioEstudiante" <<repository>> {
    - coleccion: ListaEstudiantes
    + existeId(id: String): boolean
    + guardar(estudiante: Estudiante): void
    + buscarPorId(id: String): Estudiante
+ actualizar(estudiante: Estudiante): void
    + eliminar(id: String): void + listarTodos():
ColeccionEstudiantes
}
interface "IteradorEstudiante" <<iterator>> {
    + primero()
    + haySiguiente(): boolean
    + siguiente(): Estudiante
    + actual(): Estudiante
}
interface "ColeccionEstudiantes" <<aggregate>> {
    + crearIterador(): IteradorEstudiante
    + agregar(estudiante: Estudiante)
    + eliminar(id: String)
    + obtener(posicion: int): Estudiante
    + cantidad(): int }
class
"ListaEstudiantes"
<<concreteAggregate>> {
- estudiantes: List<Estudiante>
    + crearIterador(): IteradorEstudiante
    + agregar(estudiante: Estudiante)
    + eliminar(id: String)
    + obtener(posicion: int): Estudiante
    + cantidad(): int
} class "IteradorListaEstudiantes" <<concreteIterator>>
{

```

```

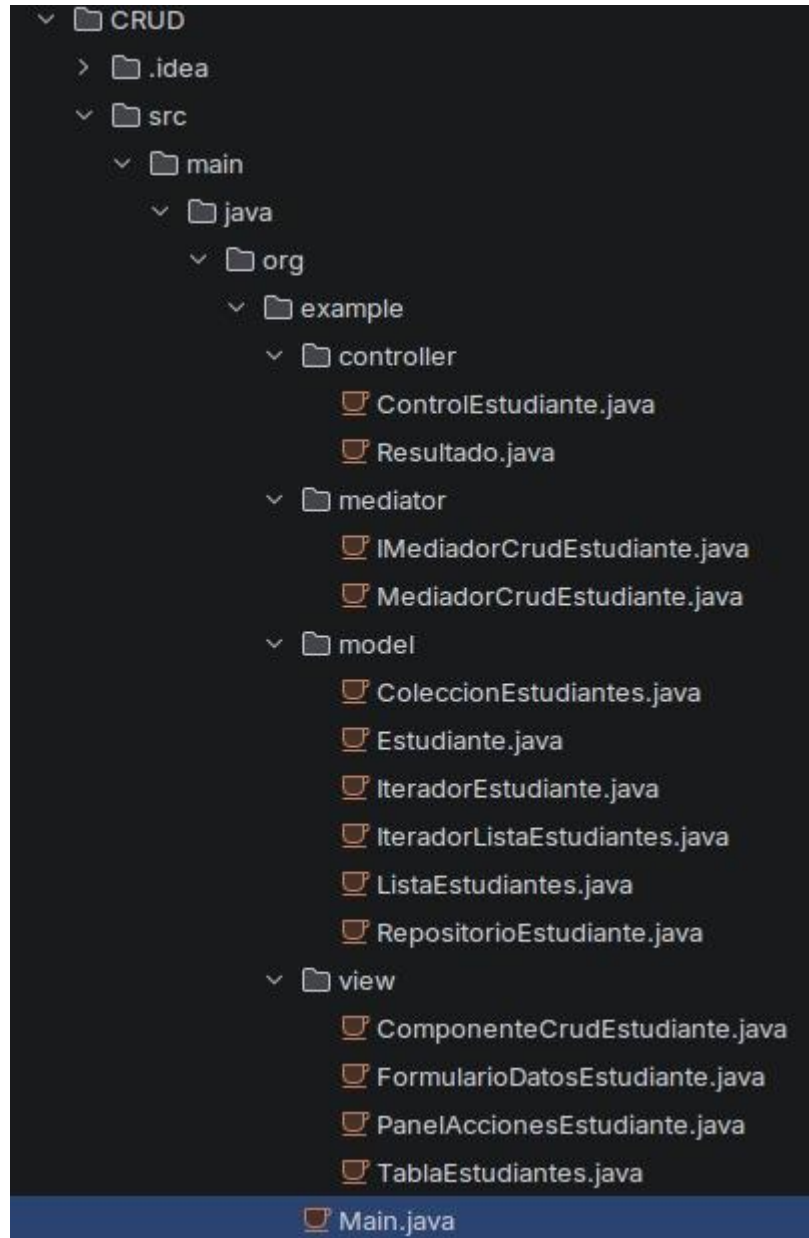
- lista: ListaEstudiantes
- posicion: int
  + primero()
  + haySiguiente(): boolean
  + siguiente(): Estudiante
  + actual(): Estudiante
}
class "Estudiante" <<entity>> {
- id: String
- nombre: String
- edad: int
  + Estudiante(id: String, nombre: String, edad:
int) + getId(): String
  + getNombre(): String
  + getEdad(): int
} class
"Resultado" {
- exito: boolean
- mensaje: String
  + esExitoso(): boolean
  + getMensaje(): String }
FormularioDatosEstudiante --|>
ComponenteCrudEstudiante
PanelAccionesEstudiante --|>
ComponenteCrudEstudiante
TablaEstudiantes --|>
ComponenteCrudEstudiante
ComponenteCrudEstudiante -->
IMediadorCrudEstudiante : notifica eventos
MediadorCrudEstudiante ..|>
IMediadorCrudEstudiante
MediadorCrudEstudiante o--
FormularioDatosEstudiante
MediadorCrudEstudiante o--
PanelAccionesEstudiante
MediadorCrudEstudiante o--
TablaEstudiantes
PanelAccionesEstudiante -->
IMediadorCrudEstudiante : clickMostrarTodo()
MediadorCrudEstudiante -->
ControlEstudiante : solicita mostrar todos
ControlEstudiante -->
RepositorioEstudiante : obtiene colección
RepositorioEstudiante o-- ListaEstudiantes : almacena
ListaEstudiantes ..|> ColeccionEstudiantes
IteradorListaEstudiantes ..|>
IteradorEstudiante
ListaEstudiantes o-- "0..*" Estudiante : contiene
ListaEstudiantes -->
IteradorListaEstudiantes : crea
IteradorListaEstudiantes -->
ListaEstudiantes : recorre
ColeccionEstudiantes --> IteradorEstudiante
: crearIterador() ControlEstudiante -->
IteradorEstudiante : retorna
MediadorCrudEstudiante -->
TablaEstudiantes : envía iterador TablaEstudiantes

```



```
--> IteradorEstudiante : recorre  
para mostrar  
ControlEstudiante --> Resultado  
RepositorioEstudiante --> Estudiante : busca/actualiza  
@enduml
```

### Implementacion en proyecto CRUD estudiante:



```
CRUD
├── .idea
├── src
│   ├── main
│   │   ├── java
│   │   │   └── org
│   │   │       └── example
│   │           ├── controller
│   │           │   ├── ControlEstudiante.java
│   │           │   └── Resultado.java
│   │           ├── mediator
│   │           │   ├── IMediatorCrudEstudiante.java
│   │           │   └── MediatorCrudEstudiante.java
│   │           ├── model
│   │           │   ├── ColeccionEstudiantes.java
│   │           │   ├── Estudiante.java
│   │           │   ├── IteradorEstudiante.java
│   │           │   └── IteradorListaEstudiantes.java
│   │           ├── ListaEstudiantes.java
│   │           ├── RepositorioEstudiante.java
│   │           └── view
│   │               ├── ComponenteCrudEstudiante.java
│   │               ├── FormularioDatosEstudiante.java
│   │               ├── PanelAccionesEstudiante.java
│   │               └── TablaEstudiantes.java
│   │               └── Main.java
│   ├── target
│   ├── .gitignore
│   └── pom.xml
└── src > main > java > org > example > model > IteradorListaEstudiantes.java > {} org.example.model
1 package org.example.model;
2
3 public class IteradorListaEstudiantes implements IteradorEstudiante {
4     private ListaEstudiantes lista;
5     private int posicion = 0;
6
7     public IteradorListaEstudiantes(ListaEstudiantes lista) {
8         this.lista = lista;
9     }
10
11     @Override
12     public void primero() {
13         posicion = 0;
14     }
15
16     @Override
17     public boolean haySiguiente() {
18         return posicion < lista.cantidad();
19     }
20
21     @Override
22     public Estudiante siguiente() {
23         return lista.obtener(posicion++);
24     }
25
26     @Override
27     public Estudiante actual() {
28         if (posicion == 0) return null;
29         return lista.obtener(posicion - 1);
30     }
31 }
```

```
CRUD
├── .idea
├── src
│   ├── main
│   │   ├── java
│   │   │   └── org
│   │   │       └── example
│   │           ├── controller
│   │           │   ├── ControlEstudiante.java
│   │           │   └── Resultado.java
│   │           ├── mediator
│   │           │   ├── IMediatorCrudEstudiante.java
│   │           │   └── MediatorCrudEstudiante.java
│   │           ├── model
│   │           │   ├── ColeccionEstudiantes.java
│   │           │   ├── Estudiante.java
│   │           │   ├── IteradorEstudiante.java
│   │           │   └── IteradorListaEstudiantes.java
│   │           ├── ListaEstudiantes.java
│   │           ├── RepositorioEstudiante.java
│   │           └── view
│   │               ├── ComponenteCrudEstudiante.java
│   │               ├── FormularioDatosEstudiante.java
│   │               ├── PanelAccionesEstudiante.java
│   │               └── TablaEstudiantes.java
│   │               └── Main.java
│   ├── target
│   ├── .gitignore
│   └── pom.xml
└── src > main > java > org > example > mediator > MediatorCrudEstudiante.java > {} org.example.mediator
11 public class MediatorCrudEstudiante implements IMediatorCrudEstudiante {
12     private FormularioDatosEstudiante formulario;
13     private TablaEstudiantes tabla;
14     private PanelAccionesEstudiante acciones;
15     private ControlEstudiante control;
16
17     public MediatorCrudEstudiante(ControlEstudiante control, FormularioDatosEstudiante formulario,
18                                 TablaEstudiantes tabla, PanelAccionesEstudiante acciones) {
19         this.control = control;
20         this.formulario = formulario;
21         this.tabla = tabla;
22         this.acciones = acciones;
23         if (this.acciones != null) this.acciones.setMediator(this);
24         if (this.tabla != null) this.tabla.setMediator(this);
25     }
26
27     @Override
28     public void agregar() {
29         Resultado r = control.agregarEstudiante(formulario.obtenerId(), formulario.obtenerNombre(), formulario.obtenerEdad());
30         formulario.mostrarMensaje(r.getMensaje());
31         mostrarTodo();
32     }
33
34     @Override
35     public void actualizar() {
36         Resultado r = control.actualizarEstudiante(formulario.obtenerId(), formulario.obtenerNombre(), formulario.obtenerEdad());
37         formulario.mostrarMensaje(r.getMensaje());
38         mostrarTodo();
39     }
40
41     @Override
42     public void eliminar() {
43         Resultado r = control.eliminarEstudiante(formulario.obtenerId());
44         formulario.mostrarMensaje(r.getMensaje());
45         mostrarTodo();
46     }
47
48     @Override
49     public void mostrarTodo() {
50         IteradorEstudiante it = control.mostrarTodos();
51         tabla.mostrarTabla(it);
52     }
53 }
```

CRUD Estudiantes

ID:

3

Nombre:

Paul

Edad:

23

Agregar

Actualizar

Eliminar

Mostrar Todo

| ID | Nombre | Edad |
|----|--------|------|
| 1  | Diana  | 23   |
| 2  | Leo    | 23   |
| 3  | Paul   | 23   |